

LLM-Based Intelligent System for Operational Failure Diagnosis in AWS Cloud Environments

Luis Felipe Patiño López  0009-0004-2658-6799 ^{1,*} and César Andrey Perdomo Charry  0000-0001-7310-4618 ¹

¹ Maestría en Ciencias de la Información y las Comunicaciones, Universidad Distrital Francisco José De Caldas, Bogotá 110231, Colombia

Abstract

The increasing complexity of cloud-based applications deployed on Amazon Web Services (AWS) poses significant challenges for timely identification and diagnosis of operational failures. Current diagnostic processes rely heavily on manual analysis of observability data, including metrics, logs, and distributed traces, which is time-consuming and error-prone. This paper presents the design, implementation, and evaluation of an intelligent diagnostic system that integrates AWS-native observability services (CloudWatch, X-Ray) with a large language model (Anthropic Claude Sonnet) accessed through Amazon Bedrock, using a Retrieval-Augmented Generation (RAG) pattern. A preliminary benchmark comparing three language models (Claude Sonnet, GPT-4o, and Llama 3 70B) justified the model selection based on diagnostic precision, response latency, and native AWS integration. The system was validated on five controlled failure scenarios and evaluated by three professionals with experience in AWS infrastructure operations. Results demonstrate a 62.7% reduction in mean time to diagnosis (MTTD) compared to manual analysis, 80% precision in root cause identification, an F1-score of 0.78, and a hallucination rate of 2.7% mitigated by RAG contextual grounding. These findings suggest that integrating language models with observability data is a viable approach for supporting operational failure diagnosis in cloud environments.

Keywords: AIOps; Amazon Web Services; cloud computing; failure diagnosis; large language models; observability; Retrieval-Augmented Generation

1. Introduction

The operation of applications in cloud computing environments has become a critical component for organizations that depend on highly available, scalable, and distributed systems. On platforms such as Amazon Web Services (AWS), modern architectures integrate multiple services and components that interact dynamically, increasing operational complexity and hindering the timely identification of failures when incidents occur in production [1,2]. Operational failure diagnosis largely depends on manual analysis of metrics, logs, and events generated by infrastructure and applications, commonly collected through observability tools such as Amazon CloudWatch [3]. This process requires technical teams to review large volumes of data from different sources, correlate dispersed information, and apply expert knowledge to infer possible root causes [4]. Chen et al. [5], in an empirical study of over 30,000 incidents in Microsoft cloud services, found that diagnosis consumes on average more than 70% of the total resolution time, highlighting the need for tools that assist this process. In this context, the AIOps (Artificial Intelligence for IT

Received:

Accepted:

Published:

Copyright: © 2026 by the author.

Submitted to *Appl. Sci.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

Operations) paradigm has emerged as an approach to automate and improve IT operations functions [6]. Notaro et al. [7] identify four functional domains: anomaly detection, root cause analysis, failure prediction, and automated remediation, with automated root cause analysis remaining one of the most relevant open challenges. Recently, the trend has evolved from rule-based and statistical approaches toward the integration of large language models (LLMs) and agentic workflows for operational diagnosis [8,9]. Ahmed et al. [8] evaluated the use of LLMs for semantic understanding of software logs, demonstrating that they outperform traditional representation methods in classification and anomaly detection tasks. Chen et al. [9] proposed a framework for LLM-assisted cloud incident diagnosis that combines automated collection of observability signals with the reasoning capability of LLMs, achieving significant reductions in diagnosis time. These advances place the present work at the convergence of cloud observability, AIOps, and generative language models. This paper describes the design, implementation, and evaluation of an intelligent system that integrates AWS observability services with a large language model (Anthropic Claude Sonnet, accessed through Amazon Bedrock) to support operational failure diagnosis. The system employs a Retrieval-Augmented Generation (RAG) pattern that structures collected observability information and provides it as context to the model for generating root cause hypotheses. A preliminary comparative evaluation among three language models is included, along with validation on five controlled failure scenarios contrasting assisted diagnosis with manual analysis.

2. Materials and Methods

2.1. Methodological Approach

The project follows a Design Science Research (DSR) approach [10], oriented toward designing, building, and evaluating technological artifacts as responses to problems identified in real contexts. The methodology was structured in five sequential phases: (1) problem analysis and environment characterization, (2) architecture design, (3) system implementation, (4) technical validation and evaluation, and (5) results analysis and conclusions.

2.2. System Architecture

The proposed architecture is organized into three functional layers. The collection layer uses Amazon CloudWatch for metric and log ingestion and AWS X-Ray for distributed traces. The processing layer, implemented through AWS Lambda functions, queries CloudWatch Logs Insights and CloudWatch Metrics APIs, structuring data into a contextualized format. The intelligent analysis layer employs Amazon Bedrock to access the LLM, implementing a RAG pattern where observability information is provided as context for hypothesis generation. The diagnostic flow is triggered upon detection of an abnormal condition through CloudWatch alarms. An orchestrator Lambda function automatically collects relevant observability data within a configurable time window (default: 15 minutes before and 5 minutes after the event), structures them into a contextualized prompt, and sends them to the model. The system generates a technical summary of the incident, a prioritized list of root cause hypotheses with supporting evidence, and mitigation recommendations.

2.3. Prompt Design and RAG Strategy

Prompt construction constitutes a critical system component. A three-block prompt structure was designed: (i) the system prompt, which defines the model's role as an expert SRE analyst and establishes explicit constraints to avoid hallucinations, instructing the model to base its hypotheses exclusively on provided evidence; (ii) the operational context block, containing structured observability data (metrics in tabular format, logs filtered by

severity, and traces with annotated latencies); and (iii) the instruction block, which requests a technical summary, prioritized hypotheses with supporting evidence, and recommended actions. Table 1 presents the prompt structure used. The retrieval strategy operates through parameterized queries to CloudWatch Logs Insights and CloudWatch Metrics APIs. Logs are filtered by severity level (ERROR and WARN) and limited to the 200 most recent records within the time window. Metrics are retrieved with 1-minute granularity for CPU, memory, network latency, and error rates. This structured text approach was preferred over vector embeddings due to lower implementation complexity and direct compatibility with the evaluated models’ context windows.

Table 1. Prompt structure of the intelligent diagnostic system.

Block	Content	Purpose
System Prompt	Role: Expert SRE analyst. Constraints: respond only based on provided evidence. Do not speculate beyond data.	Ground the model to context and reduce hallucinations
Operational Context	Metrics (CPU, memory, latency, errors), filtered logs (ERROR, WARN, last 20 min), X-Ray traces with per-service latencies.	Provide structured evidence for analysis
Instruction	Generate: (1) technical incident summary, (2) 3–5 prioritized root cause hypotheses with evidence, (3) recommended mitigation actions.	Structure the model output

2.4. Language Model Comparative Evaluation

Prior to system validation, a preliminary evaluation was conducted comparing three language models under identical prompting and observability context conditions: Anthropic Claude Sonnet (via Amazon Bedrock), GPT-4o (via Azure OpenAI), and Llama 3 70B (via Amazon Bedrock with open-source model). Selection criteria focused on three dimensions: diagnostic precision (correct root cause identification in 3 pilot scenarios), response latency (time from prompt submission to complete response reception), and effective context window (ability to process extensive logs without degradation). Table 2 presents the results of this evaluation. Claude Sonnet was selected for its balance between diagnostic precision, competitive latency, and native integration with the AWS ecosystem through Amazon Bedrock, which simplifies the architecture and reduces dependency on external services. The 200K-token context window proved particularly relevant for scenarios with high log volumes, where Llama 3 70B presented limitations due to its 8K-token window.

Table 2. Preliminary comparative evaluation of language models.

Model	Precision	Latency (s)	Context	Cost/1K (USD)	AWS Integr.
Claude Sonnet	3/3 (100%)	4.2	200K	0.003/0.015	Native (Bedrock)
GPT-4o	3/3 (100%)	5.8	128K	0.005/0.015	External (API)
Llama 3 70B	2/3 (67%)	7.1	8K	0.00265/0.0035	Bedrock (custom)

2.5. Evaluation Scenarios

Five controlled failure scenarios were defined, executed in an isolated test environment on AWS: (E1) compute resource exhaustion on EC2 instances, (E2) timeout errors in

Lambda invocations, (E3) latency degradation in RDS database services, (E4) connectivity failures between microservices deployed on Amazon ECS, and (E5) configuration errors in IAM/VPC security policies. Each scenario was documented with reproducible initial conditions and a known root cause as evaluation ground truth.

2.6. Evaluation Protocol

The comparative evaluation was conducted with a panel of three evaluators with a minimum of two years of professional experience in DevOps or SRE roles in AWS environments. A within-subjects design with order counterbalancing was employed: each evaluator analyzed the five scenarios first without system access (manual baseline) and subsequently with system access (assisted diagnosis), or vice versa, to mitigate learning effects. Evaluation metrics defined a priori were: (i) mean time to diagnosis (MTTD), measured from incident notification to root cause identification; (ii) precision, calculated as the proportion of scenarios where the root cause was correctly identified; (iii) F1-score of generated hypotheses; (iv) MTTD reduction percentage; and (v) false positive and model hallucination rate.

3. Results

3.1. Mean Time to Diagnosis

Table 3 presents the comparative results for the five evaluated scenarios. The assisted system achieved an average reduction of 62.7% in MTTD compared to manual analysis. Average MTTD decreased from 25.1 minutes in manual diagnosis to 8.8 minutes with the assisted system. The greatest reductions were observed in scenarios E2 (Lambda timeout, 67.2%) and E4 (microservice connectivity, 66.5%), where the system’s ability to rapidly correlate multiple information sources proved especially useful.

Table 3. Comparative results by failure scenario.

Scenario	Manual MTTD (min)	Assisted MTTD (min)	Reduction (%)	Precision	F1-Score
E1: CPU Exhaustion (EC2)	24.3	8.7	64.2	1.00	0.83
E2: Timeout (Lambda)	17.7	5.8	67.2	1.00	0.86
E3: Latency (RDS)	28.5	10.2	64.2	0.67	0.73
E4: Connectivity (ECS)	21.8	7.3	66.5	0.67	0.71
E5: Security Config.	33.2	11.8	64.5	0.67	0.75
Average	25.1	8.8	62.7	0.80	0.78

3.2. Precision and Root Cause Hypotheses

The system achieved a global precision of 80% in correct root cause identification. In scenarios E1 and E2, the root cause was correctly identified in 100% of evaluations, attributable to well-defined failure patterns with clear observability signals (CPU spikes, explicit timeout errors in logs). In scenarios E3, E4, and E5, precision was 67%: the system identified the root cause as part of the presented hypotheses but did not always prioritize it as the primary hypothesis, particularly when multiple concurrent anomalies coexisted. The average F1-score was 0.78, indicating an adequate balance between relevance and completeness of hypotheses.

3.3. System Latency

The average diagnostic report generation time, measured from CloudWatch alarm activation to summary delivery to the operator, was 47 seconds. This time breaks down as: observability data collection (18 s), prompt structuring (3 s), LLM inference (22 s), and response formatting (4 s). This latency is acceptable for operational diagnostic scenarios, where the observed average manual MTTD was 25.1 minutes.

3.4. False Positive and Hallucination Analysis

System-generated hypotheses that did not correspond to actual environment conditions (false positives) or that referenced nonexistent components, services, or events (hallucinations) were recorded. Out of a total of 75 hypotheses generated across 15 evaluations (5 scenarios $\times$ 3 evaluators), 9 irrelevant hypotheses (12%) and 2 hallucination cases (2.7%) were identified, where the model referenced services not present in the test architecture. The RAG strategy contributed to mitigating hallucinations by grounding model responses to the observability evidence provided in the context. The system prompt with explicit constraints (“respond only based on provided evidence”) reduced hallucinations compared to pilot tests without this constraint, where the observed rate was 8.5%.

3.5. Qualitative Analysis

Evaluators highlighted the utility of generated technical summaries, particularly the temporal correlation between events from different services and the structured presentation of evidence. A limitation was identified in scenarios with multiple concurrent anomalies (E4, E5), where the system tends to prioritize the most evident cause without sufficiently exploring secondary interactions between components. All three evaluators agreed that the system functions as an effective complement to expert judgment, accelerating the initial triage phase and reducing the diagnostic search space.

4. Discussion

4.1. Comparison with Related Work

The obtained results are consistent with previous findings in the literature. Chen et al. [9] reported similar reductions in diagnosis time when combining LLMs with observability data in cloud environments. The 80% precision aligns with levels reported by Dang et al. [11] in AIOps systems implemented at Microsoft. The 2.7% hallucination rate is lower than those reported in general LLM applications, attributed to the RAG pattern effect and restrictive system prompt design, consistent with findings by Lewis et al. [19] on RAG effectiveness for reducing ungrounded generation.

4.2. Cost–Benefit Analysis

The average operational cost per assisted diagnosis was estimated at USD 0.12, considering token consumption in Amazon Bedrock (approximately 8000 input tokens and 2000 output tokens per query) and Lambda function execution costs. In contrast, the estimated manual diagnosis cost, calculated from the average MTTR of 25.1 minutes and a reference hourly rate of USD 40 for an SRE/DevOps engineer, amounts to approximately USD 16.7 per incident. This two-order-of-magnitude difference suggests significant economic viability, although it should be noted that the assisted system does not completely eliminate human intervention.

4.3. Limitations and Threats to Validity

Internal validity. The reduced sample size ($N = 3$ evaluators) constitutes the primary threat to internal validity. While the within-subjects design with counterbalancing partially mitigates learning effects and individual differences, the sample does not allow for statistical significance tests with adequate power. Future work should expand the evaluation to a panel of 10 to 15 professionals to enable significance tests (Student’s t or Wilcoxon) on MTTR results. **External validity.** The evaluation was conducted on five controlled scenarios with a single predominant root cause. The variability and complexity of incidents in real production environments, including cascade failures and multiple concurrent causes, could reduce the observed precision. The absence of a multi-causal

failure scenario limits conclusions about the system's discernment capability in complex component interactions. **Instrumentation dependency.** System effectiveness is intrinsically linked to the quality of collected observability data. The absence of AWS X-Ray traces or insufficiently detailed logs drastically limits the model's ability to identify root causes in complex scenarios. Applications with poor instrumentation may not benefit from the proposed approach. **Context window and costs.** The RAG pattern implementation implies that, given massive log volumes, there is a risk of exceeding the LLM context window or significantly increasing operational costs through token consumption in Amazon Bedrock. The severity-based filtering and time-limit strategy partially mitigates this risk but may exclude relevant information in long-duration incidents. **Controlled environment vs. production.** Test scenarios were executed in an isolated environment without significant concurrent workload. The operational noise characteristic of production environments (logs from multiple applications, metrics from unrelated services, spurious alerts) could degrade diagnostic precision by introducing distracting information into the model context.

5. Conclusions

This paper presented the design, implementation, and evaluation of an intelligent system for supporting operational failure diagnosis in AWS-based cloud environments. Integrating platform-native observability services with a large language model through a RAG pattern enabled generating technical summaries, correlations, and root cause hypotheses from heterogeneous metrics, logs, and event data, facilitating the diagnostic triage phase. The preliminary comparative evaluation among three language models justified the selection of Anthropic Claude Sonnet for its balance between precision, latency, and native AWS integration. Validation on five controlled scenarios demonstrated a 62.7% reduction in MTTD, 80% precision, and an F1-score of 0.78, with a 2.7% hallucination rate mitigated by prompt design and RAG strategy. These findings contribute to the AIOps body of knowledge by demonstrating the viability of using commercial LLMs accessed through managed services (Amazon Bedrock) to address operational complexity challenges in cloud environments. System effectiveness critically depends on observability instrumentation quality and proper configuration of analysis time windows. Future research directions include: (i) expanding the evaluation to a panel of 10 to 15 professionals with statistical significance tests; (ii) including cascade failure and multiple concurrent cause scenarios; (iii) transitioning toward agentic workflows enabling automated remediation of simple failures; and (iv) exploring fine-tuning techniques on open-source models specialized in the site reliability engineering (SRE) domain, aiming to optimize operational costs and improve performance in multi-causal failure scenarios. **Author Contributions:** Conceptualization, L.F.P.L.

and C.A.P.C.; methodology, L.F.P.L.; software, L.F.P.L.; validation, L.F.P.L.; formal analysis, L.F.P.L.; investigation, L.F.P.L.; resources, L.F.P.L. and C.A.P.C.; data curation, L.F.P.L.; writing—original draft preparation, L.F.P.L.; writing—review and editing, C.A.P.C.; visualization, L.F.P.L.; supervision, C.A.P.C.; project administration, C.A.P.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. Infrastructure costs were funded by the first author through a personal AWS account.

Data Availability Statement: The observability data generated during the controlled failure scenarios are available from the corresponding author upon reasonable request. The test environment configurations and evaluation protocols are described in the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Mell, P.; Grance, T. The NIST Definition of Cloud Computing. *National Institute of Standards and Technology, Special Publication 800-145*, **2011**. 238
2. Newman, S. *Building Microservices: Designing Fine-Grained Systems*, 2nd ed.; O'Reilly Media: Sebastopol, CA, USA, 2021. 239
3. Amazon Web Services. *Amazon CloudWatch User Guide*. Available online: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/> (accessed on 15 March 2026). 240
4. Sridharan, C. *Distributed Systems Observability*; O'Reilly Media: Sebastopol, CA, USA, 2018. 241
5. Chen, J.; He, X.; Lin, Q.; Xu, Y.; Zhang, H.; Hao, D.; Gao, F.; Xu, Z.; Dang, Y.; Zhang, D. An Empirical Investigation of Incident Triage for Online Service Systems. In Proceedings of the 42nd International Conference on Software Engineering (ICSE-SEIP '20), Seoul, Republic of Korea, 27 June–19 July 2020; pp. 111–120. 242
6. Gartner. *Market Guide for AIOps Platforms*; Gartner, Inc.: Stamford, CT, USA, 2017. 243
7. Notaro, P.; Cardoso, J.; Gerndt, M. A Systematic Mapping Study in AIOps. In Proceedings of the International Conference on Service-Oriented Computing (ICSOC 2020), Dubai, United Arab Emirates, 14–17 December 2020; Springer: Cham, Switzerland, 2021; pp. 110–123. 244
8. Ahmed, T.; Devanbu, P.; Hellendoorn, V. Recommending Root-Cause and Mitigation Steps for Cloud Incidents Using Large Language Models. In Proceedings of the 45th International Conference on Software Engineering (ICSE '23), Melbourne, Australia, 14–20 May 2023; IEEE/ACM. 245
9. Chen, X.; Lin, Q.; Zhang, D.; Zhang, H. LLM-Assisted Diagnosis of Cloud Incidents Using Observability Data. In Proceedings of the IEEE International Conference on Cloud Engineering (IC2E), Chicago, IL, USA, 2024; IEEE. 246
10. Hevner, A.R.; March, S.T.; Park, J.; Ram, S. Design Science in Information Systems Research. *MIS Quarterly* **2004**, *28*, 75–105. 247
11. Dang, Y.; Lin, Q.; Huang, P. AIOps: Real-World Challenges and Research Innovations. In Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: Companion (ICSE-Companion '19), Montreal, QC, Canada, 25–31 May 2019; pp. 4–7. 248
12. Beyer, B.; Jones, C.; Petoff, J.; Murphy, N.R. *Site Reliability Engineering: How Google Runs Production Systems*; O'Reilly Media: Sebastopol, CA, USA, 2016. 249
13. He, S.; Zhu, J.; He, P.; Lyu, M.R. A Survey on Automated Log Analysis for Reliability Engineering. *ACM Computing Surveys* **2021**, *54*, 1–37. 250
14. Soldani, J.; Brogi, A. Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. *ACM Computing Surveys* **2022**, *55*, 1–39. 251
15. Du, M.; Li, F.; Zheng, G.; Srikumar, V. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17), Dallas, TX, USA, 30 October–3 November 2017; pp. 1285–1298. 252
16. Majors, C.; Fong-Jones, L.; Miranda, G. *Observability Engineering: Achieving Production Excellence*; O'Reilly Media: Sebastopol, CA, USA, 2022. 253
17. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. *Advances in Neural Information Processing Systems* **2017**, *30*, 5998–6008. 254
18. Zhou, X.; Peng, X.; Xie, T.; Sun, J.; Ji, C.; Li, W.; Ding, D. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* **2021**, *47*, 3421–3447. 255
19. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* **2020**, *33*, 9459–9474. 256
20. Meng, Y.; Zhang, S.; Sun, Y.; Zhang, R.; Hu, Z.; Zhang, Y.; Jia, C.; Wang, Z.; Pei, D. Localizing Failure Root Causes in a Microservice through Causality Inference. In Proceedings of the IEEE/ACM 28th International Symposium on Quality of Service (IWQoS), Hang Zhou, China, 15–17 June 2020; IEEE. 257
21. Jamshidi, P.; Pahl, C.; Mendonça, N.C.; Lewis, J.; Tilkov, S. Microservices: The Journey so Far and Challenges Ahead. *IEEE Software* **2018**, *35*, 24–35. 258